Serverless Image Processing Pipeline – Step-by-Step User Manual

Nov 8, 2026 · Md Alamgir Hossain

Chapter 1 – Introduction

In this manual I show how we go from an empty AWS account to a working serverless pipeline that turns every uploaded image into a 128-pixel thumbnail. I follow the project's README step by step, with a screenshot and a short note for each step. Every screenshot comes from my real deployment in AWS region ca-central-1 (Canada Central) on November 8, 2026.

1.1 What we build

When an image lands in the **originals** S3 bucket, or a client calls the **API Gateway** endpoint, an **AWS Step Functions** state machine invokes a **Lambda** function. The function resizes the image with **Pillow** and saves the thumbnail in a second **resized** S3 bucket. **CloudWatch** keeps the logs and the execution history.

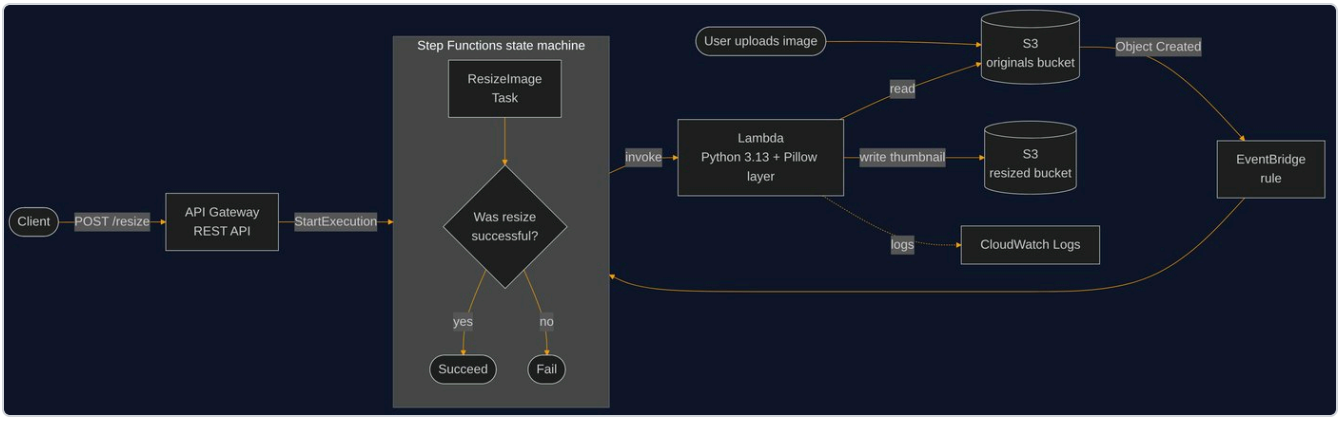


Figure 1.1 – Architecture. Two ways in (S3 upload through EventBridge, or POST /resize through API Gateway) start the same Step Functions workflow.

1.2 The workflow inside Step Functions

State	Type	What it does
ResizeImage	Task	Calls the resize Lambda; retries short-lived Lambda service errors
RecordUnexpectedError	Pass	Turns a crash or timeout into a status FAILED result
WasResizeSuccessful	Choice	Checks whether status equals SUCCEEDED
ResizeSucceeded	Succeed	The thumbnail was created
ResizeFailed	Fail	The image could not be resized

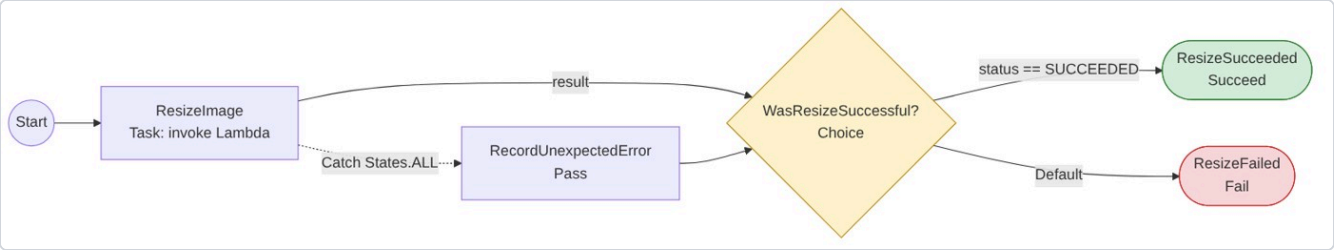


Figure 1.2 – State machine. Every run passes through the Choice state, which sends it to Succeed or Fail.

1.3 How the project meets the assignment

Assignment requirement	How this project does it
S3 bucket for original images	image-pipeline-originals-<account-id>
Lambda runs when a new image is uploaded	S3 → EventBridge → Step Functions → ResizeImageFunction
Lambda resizes the image to a thumbnail	Pillow, longest side 128 px, aspect ratio kept
Resized image stored in another bucket	image-pipeline-resized-<account-id>/thumbnails/
Step Functions orchestrates the workflow	Task, Choice, Succeed and Fail states
API Gateway endpoint triggers the state machine	POST /resize, wired straight to states:StartExecution
Pillow added to Lambda	PillowLayer, a Lambda layer built by AWS SAM

1.4 How this manual is organised

1. Chapter 2 – what we need before we start.
2. Chapter 3 – we get the source code from GitHub.
3. Chapter 4 – we deploy everything with AWS SAM.
4. Chapters 5 and 6 – we test the upload trigger, the API trigger and the failure path.
5. Chapter 7 – we check the logs and the configuration.
6. Chapter 8 – the results.
7. Chapter 9 – we fix common problems and delete everything afterwards.

Chapter 2 – Prerequisites

We need only a web browser and an AWS account: I did every step in **AWS CloudShell**, which already has the AWS CLI, the SAM CLI, Python and Git, and uses our console sign-in. The README's local-install route (AWS CLI v2, SAM CLI, Python 3.13) works the same way if we prefer our own computer.

2.1 What we need

Item	Why we need it	Notes
AWS account	Hosts S3, Lambda, Step Functions, API Gateway, EventBridge and CloudWatch	Permission to create those services and IAM roles
AWS region	Where the stack lives	I used ca-central-1 (Canada Central)
Web browser	AWS Console and CloudShell	Chrome or Edge
GitHub	Holds the source code	hossain8078/serverless-image-processing-pipeline
Test images	Something to resize	JPG, JPEG, PNG, GIF or WEBP, up to 20 MB

2.2 Before we start

1. We sign in to the AWS Console and pick our region in the top-right corner (I chose **Canada (Central)**).
2. We do **not** create access keys: CloudShell uses our console session, so no secret keys are needed.
3. If we work locally instead, we run `aws --version`, `sam --version`, `python --version` and `aws sts get-caller-identity` to confirm the tools work.

Cost note: with a few test images the project stays inside the AWS Free Tier. We delete the stack afterwards (Chapter 9) so nothing keeps running.

Chapter 3 – Get the source code

All the code lives in one public GitHub repository, so we only need its URL: `https://github.com/hossain8078/serverless-image-processing-pipeline.git`. Section 3.1 shows how I created it; anyone reusing the project can skip straight to 3.2.

3.1 How I created the repository

1. On GitHub I chose **New repository**, typed the name `serverless-image-processing-pipeline` and a short description, and kept it **Public** so the instructor can open it.
2. I left README, .gitignore and license unticked, because the project brings its own.
3. I clicked **Create repository**, then **uploading an existing file**, added the project files with their folders, and committed.

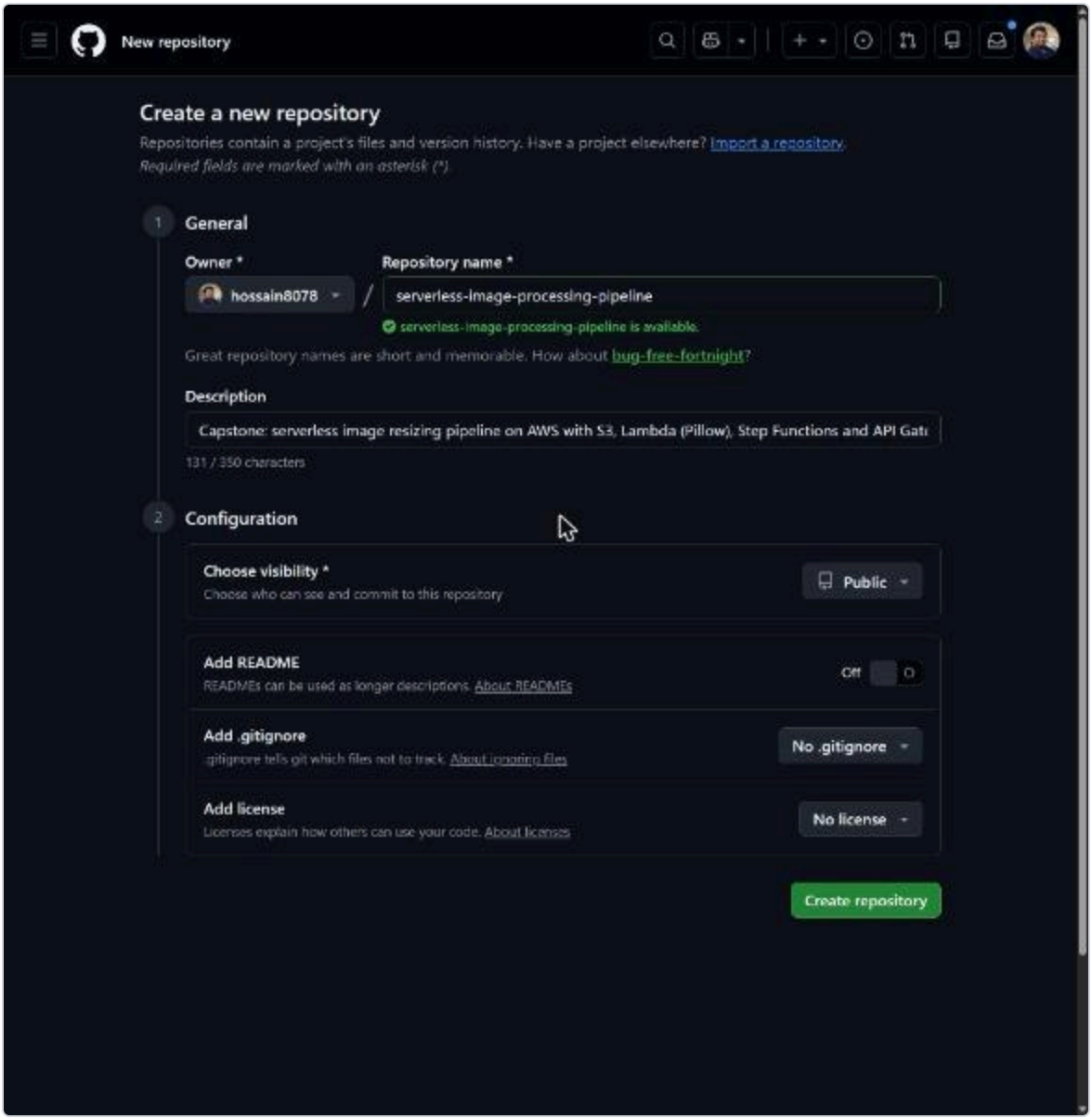


Figure 3.1 – New repository form: name, description and Public visibility filled in.

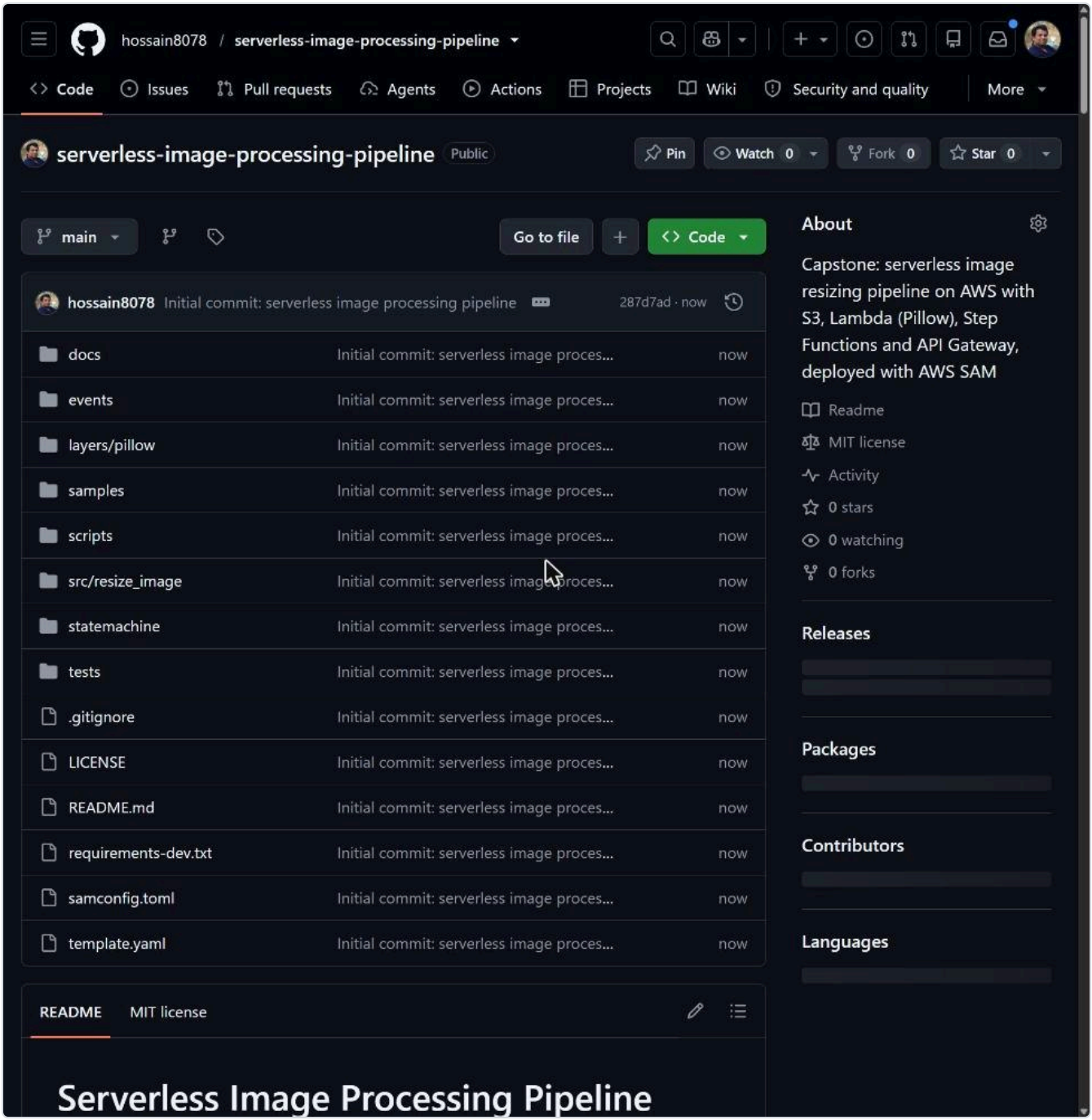


Figure 3.2 – The finished repository: source folders, README, LICENSE and the description on the right.

3.2 What is in the repository

Path	What it holds
template.yaml	AWS SAM template: every AWS resource as code
samconfig.toml	Default settings for sam deploy
statemachine/image_processing.asl.json	Step Functions definition (Amazon States Language)
src/resize_image/app.py	Lambda handler that makes the thumbnail with Pillow

Path	What it holds
layers/pillow/requirements.txt	Pillow, packaged as a Lambda layer
tests/	18 pytest unit tests; no AWS account needed
events/	Sample inputs for console testing
scripts/	End-to-end test scripts for PowerShell and bash
samples/sample.jpg	A 1600×1200 test image
docs/	Diagrams, screenshots, test results and this manual



Figure 3.3 – The state machine definition on GitHub: the ResizeImage Task, its retries and Catch, then the WasResizeSuccessful Choice.

3.3 Optional: run the unit tests

The tests stub S3, so they run on any computer with Python:

```
python -m venv .venv
pip install -r requirements-dev.txt
python -m pytest -v
```

```
$ python -m pytest -v
 ============================= test session starts ==============================
 platform linux -- Python 3.13.18, pytest-9.1.1, pluggy-1.6.0 -- /usr/local/bin/python3
 rootdir: /home/claude/serverless-image-processing-pipeline
 plugins: platformdirs-4.13.5, anyio-4.15.1
 collecting ... collected 18 items

 tests/test_app.py::test_parse_api_event_defaults_to_source_bucket PASSED [  5%]
 tests/test_app.py::test_parse_eventbridge_event PASSED                   [ 11%]
 tests/test_app.py::test_parse_s3_notification_decodes_key PASSED         [ 16%]
 tests/test_app.py::test_parse_event_requires_key PASSED                  [ 22%]
 tests/test_app.py::test_build_thumbnail_key PASSED                       [ 27%]
 tests/test_app.py::test_unsupported_extension PASSED                     [ 33%]
 tests/test_app.py::test_resize_keeps_aspect_ratio PASSED                 [ 38%]
 tests/test_app.py::test_handler_success PASSED                           [ 44%]
 tests/test_app.py::test_handler_png_with_transparency PASSED             [ 50%]
 tests/test_app.py::test_handler_eventbridge_event PASSED                 [ 55%]
 tests/test_app.py::test_handler_failures_return_failed_status[event0] PASSED [ 61%]
 tests/test_app.py::test_handler_failures_return_failed_status[event1] PASSED [ 66%]
 tests/test_app.py::test_handler_failures_return_failed_status[event2] PASSED [ 72%]
 tests/test_app.py::test_handler_failures_return_failed_status[event3] PASSED [ 77%]
 tests/test_app.py::test_handler_failures_return_failed_status[event4] PASSED [ 83%]
 tests/test_state_machine.py::test_required_state_types_exist PASSED      [ 88%]
 tests/test_state_machine.py::test_every_next_points_to_a_real_state PASSED [ 94%]
 tests/test_state_machine.py::test_task_invokes_resize_lambda PASSED      [100%]

 ============================== 18 passed in 0.11s ==============================
```

Figure 3.4 – All 18 unit tests pass: event parsing, thumbnail size, PNG transparency and every failure case.

Chapter 4 – Deploy with AWS SAM

One `sam build` and one `sam deploy` create all 15 resources in about three minutes. I ran them in CloudShell; the commands are the same as the README's Deploy section.

4.1 Open CloudShell

1. In the AWS Console we click the **CloudShell** icon (bottom-left, or the terminal icon at the top) with our region selected.
2. We close the welcome box. The terminal is ready when we see the `~ $` prompt.
3. We check the tools: `sam --version; aws --version; python3 --version; aws sts get-caller-identity --query Account --output text`

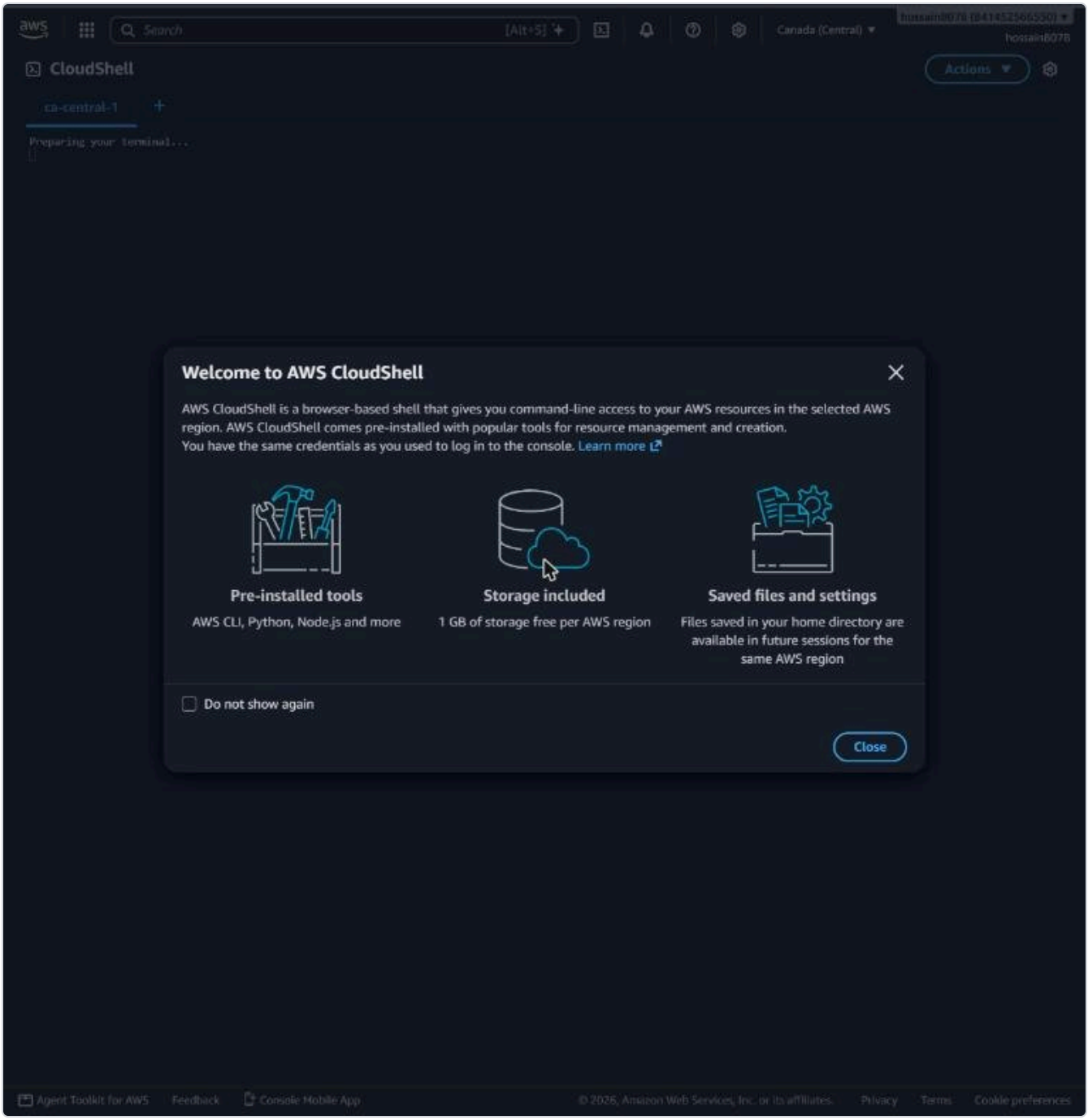


Figure 4.1 – CloudShell opens in ca-central-1 with its welcome box; we close it to reach the terminal.

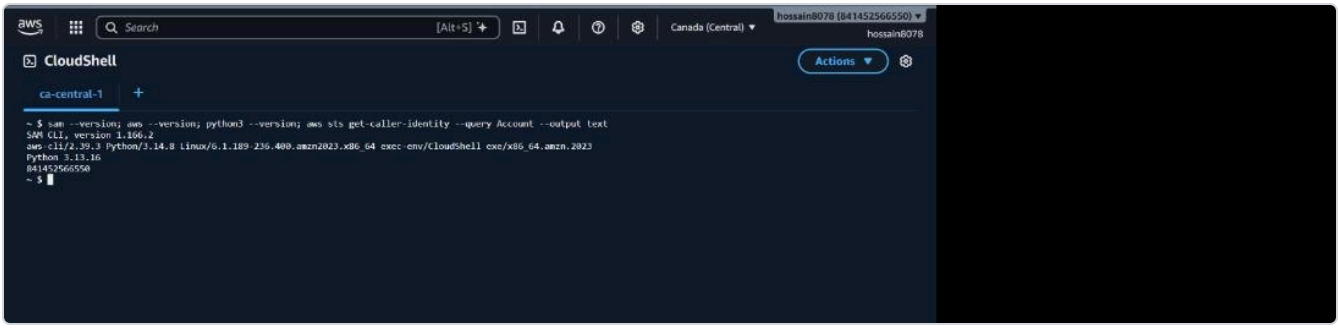


Figure 4.2 – Tool check: SAM CLI 1.166, AWS CLI 2.39, Python 3.13 and our 12-digit account ID.

4.2 Clone the repository

```
git clone https://github.com/hossain8078/serverless-image-processing-pipeline.git
cd serverless-image-processing-pipeline
ls
```

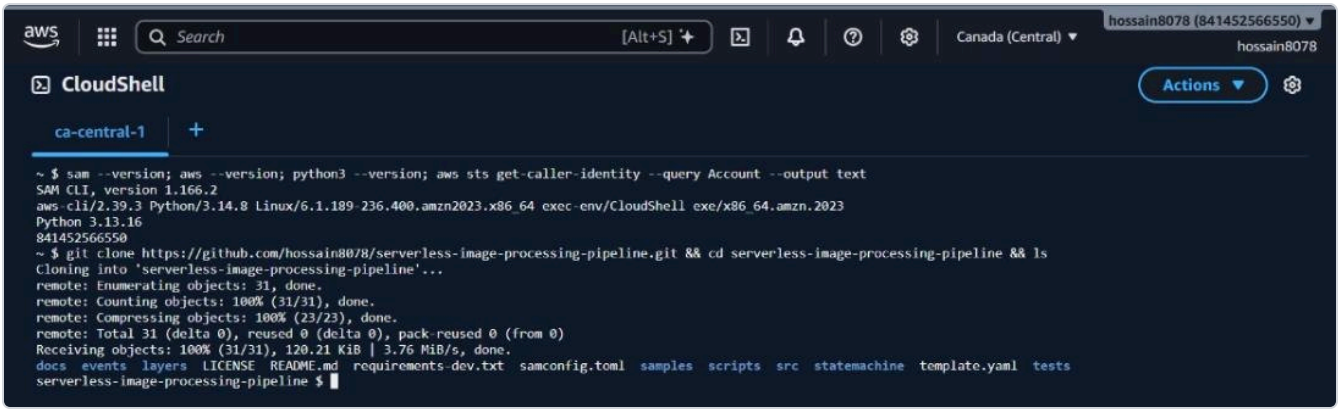


Figure 4.3 – The clone finishes and ls lists the project folders and template.yaml.

4.3 Build

```
sam build
```

SAM packages the Lambda code and builds the Pillow layer for Python 3.13. If the build fails on a local computer, we run `sam build --use-container` (it needs Docker).

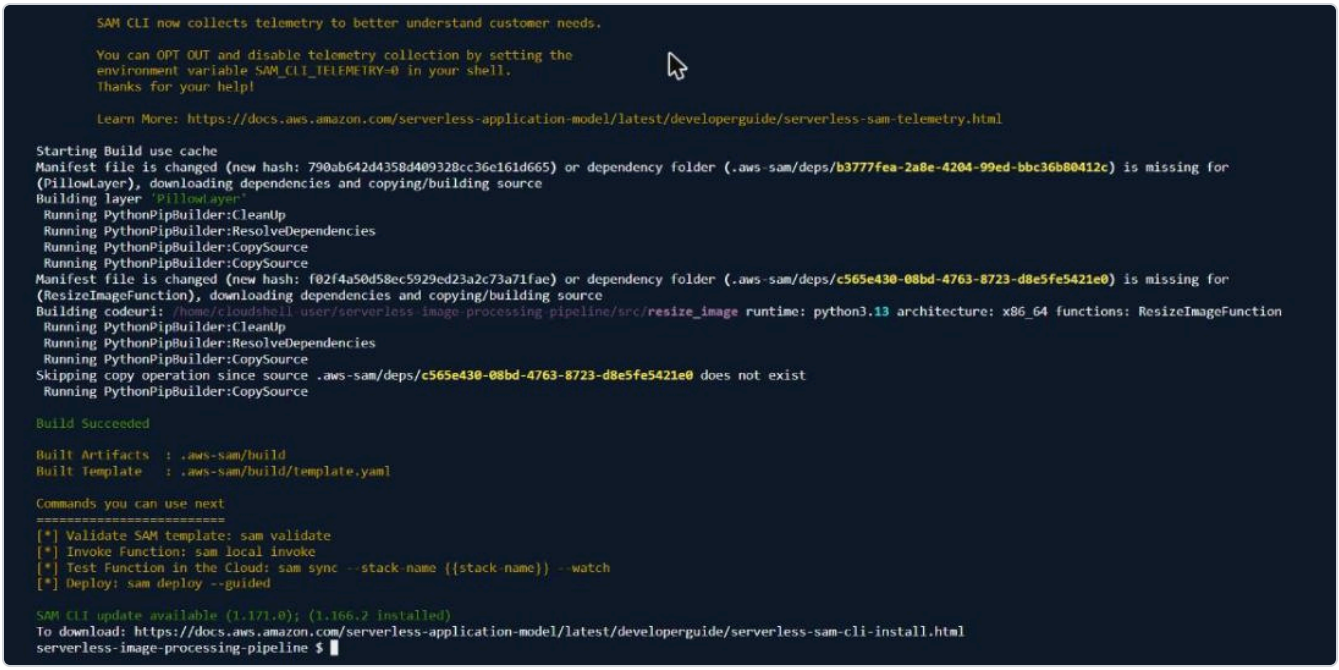


Figure 4.4 – Build Succeeded: the PillowLayer and ResizeImageFunction are built into .aws-sam/build.

4.4 Validate and deploy

```
sam validate --lint
sam deploy --stack-name image-pipeline --region ca-central-1 --capabilities CAPABILITY_IAM
--resolve-s3 --no-confirm-changeset
```

The stack name must be lowercase, because it becomes part of the bucket names. To be asked each setting instead, we run `sam deploy --guided` and answer as the README's table shows.



Figure 4.5 – The template is a valid SAM template; SAM starts creating the resources it needs.

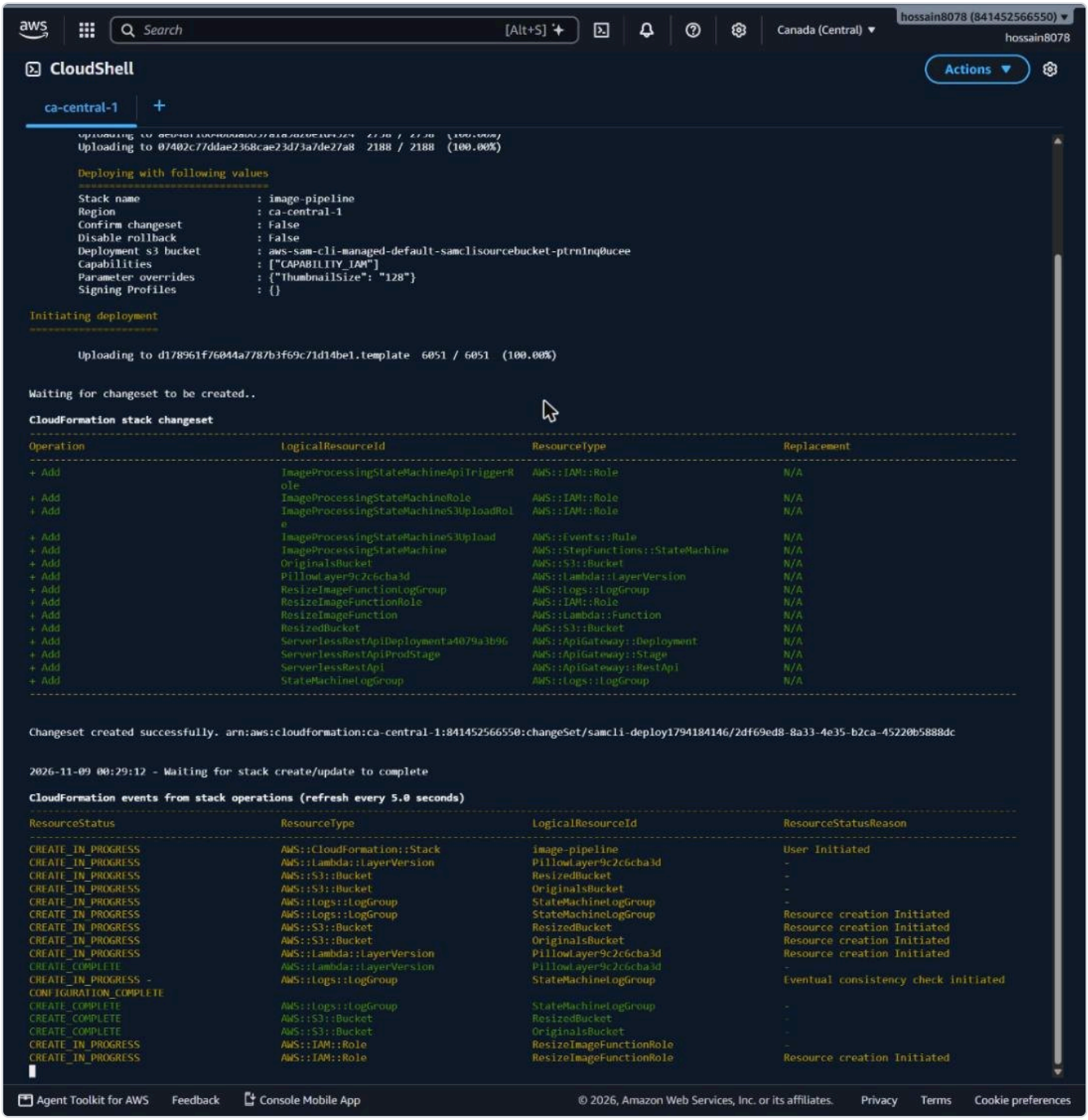


Figure 4.6 – The change set lists the 15 resources to add: buckets, Lambda, layer, roles, state machine, EventBridge rule, REST API and log groups.

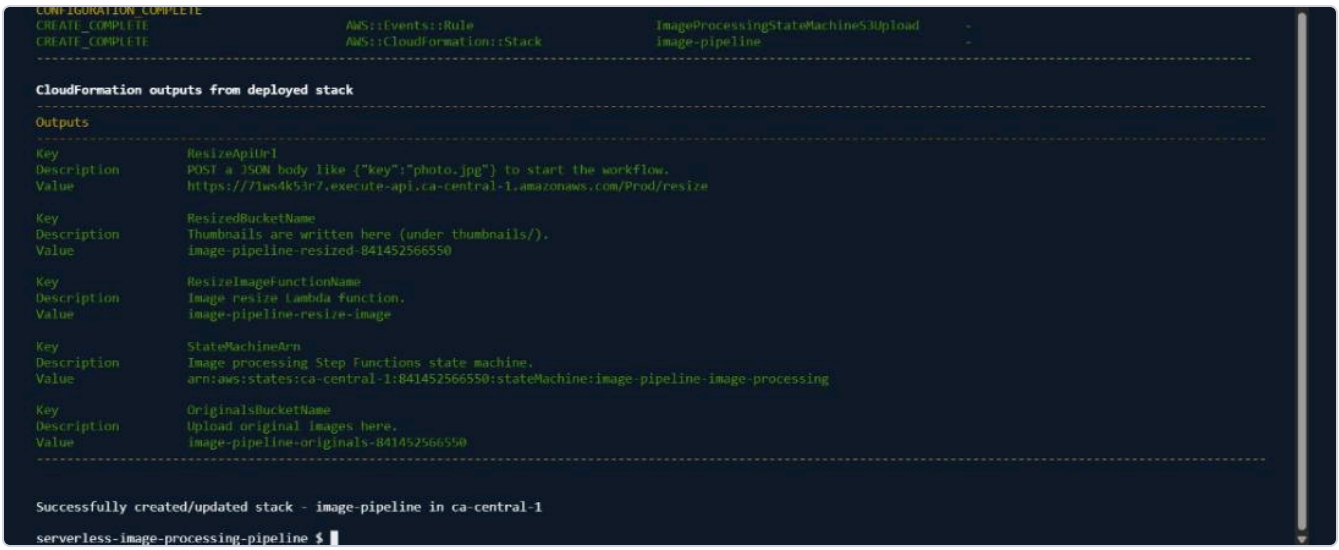


Figure 4.7 – Deploy complete: CloudFormation prints the stack outputs and "Successfully created/updated stack".

4.5 Note the outputs

Output	My value
OriginalsBucketName	image-pipeline-originals-841452566550
ResizedBucketName	image-pipeline-resized-841452566550
ResizeImageFunctionName	image-pipeline-resize-image
StateMachineArn	arn:aws:states:ca-central-1:841452566550:stateMachine:image-pipeline-image-processing
ResizeApiUrl	https://71ws4k53r7.execute-api.ca-central-1.amazonaws.com/Prod/resize

We can print them again at any time with `aws cloudformation describe-stacks --stack-name image-pipeline --query "Stacks[0].Outputs" --output table`.

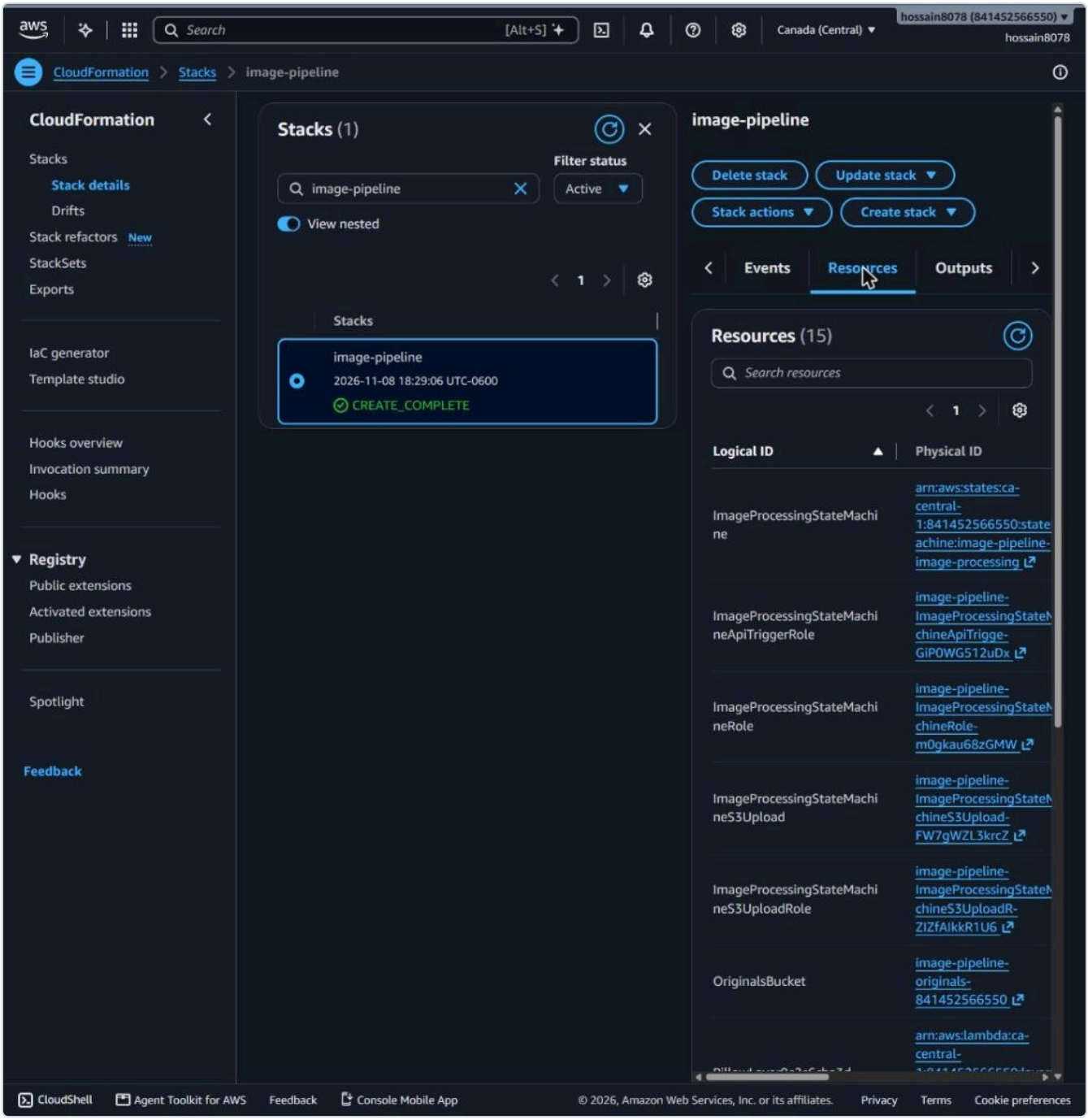


Figure 4.8 – In the CloudFormation console the image-pipeline stack shows CREATE_COMPLETE with 15 resources.

Chapter 5 – Test the automatic trigger (S3 upload)

Uploading an image is all it takes: within about 2 seconds a Step Functions run starts on its own and a thumbnail appears in the resized bucket. This is README test Option A; I used three photos named live-demo-hilltop, live-demo-school and live-demo-team.

5.1 Upload images to the originals bucket

1. We open **S3 → Buckets → image-pipeline-originals-<account-id>** and click **Upload**.
2. We click **Add files**, pick one or more images, then click **Upload** at the bottom.
3. CLI alternative: `aws s3 cp samples/sample.jpg s3://<OriginalsBucketName>/sample.jpg`

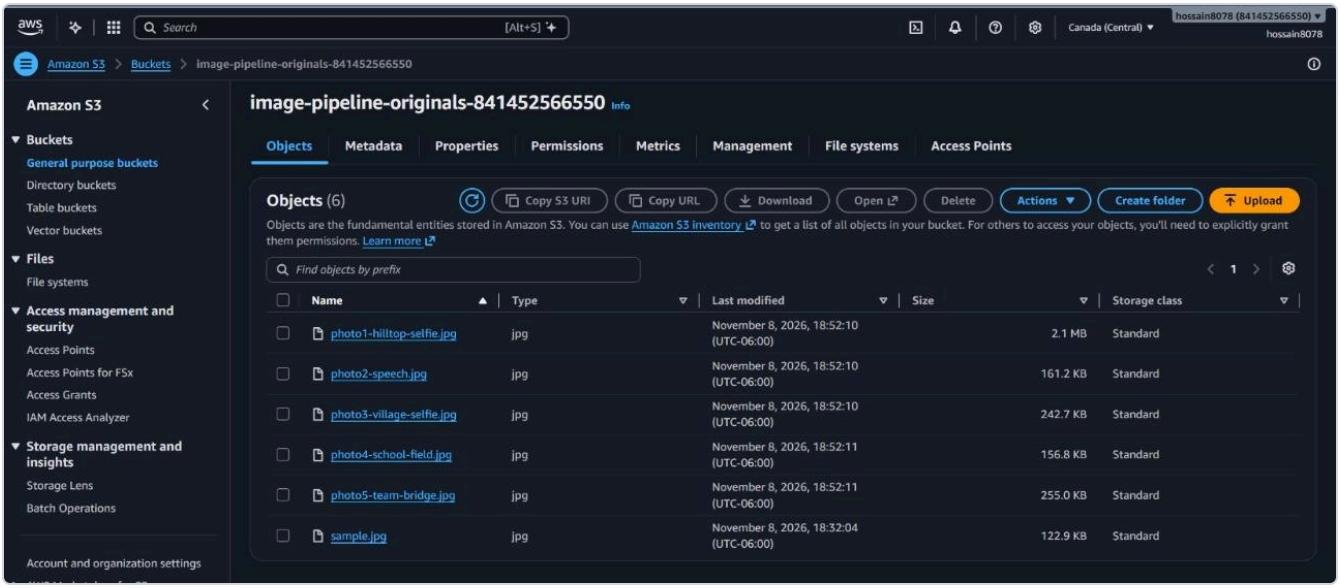


Figure 5.1 – The originals bucket before the upload; Upload is the orange button on the right.

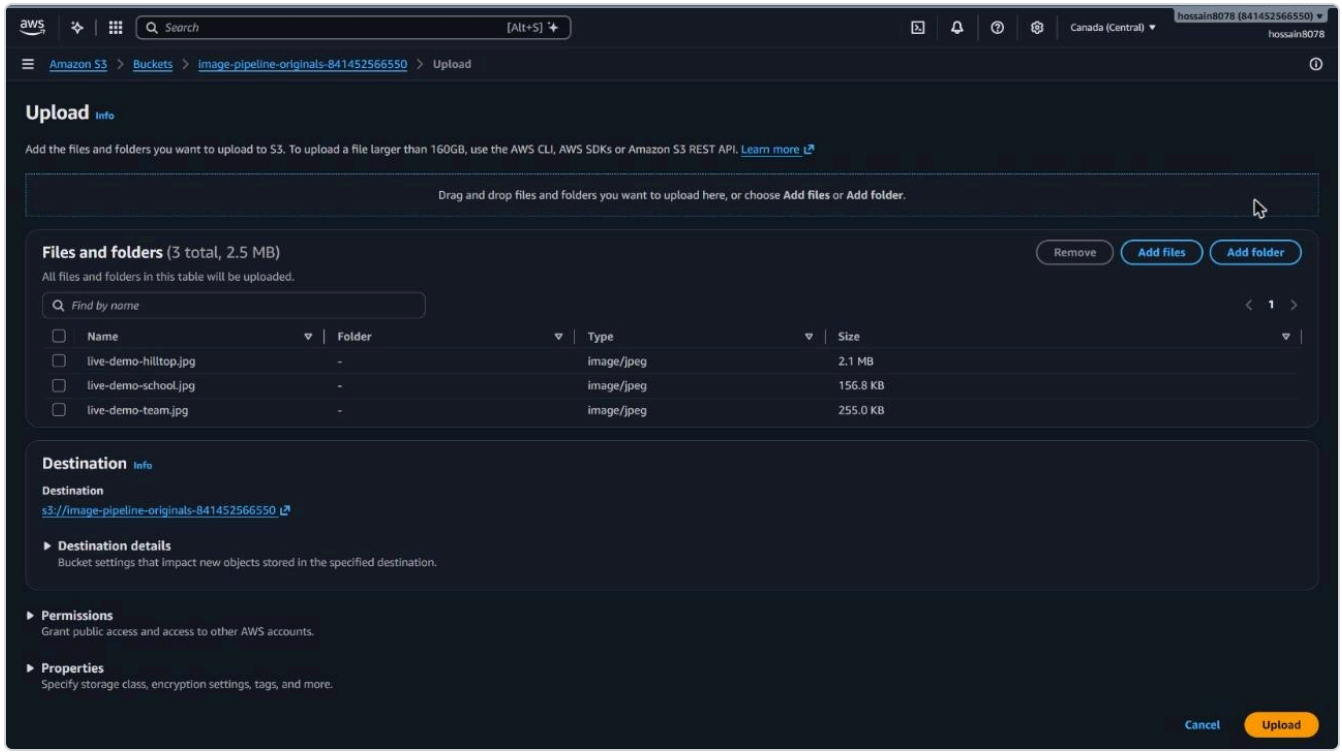


Figure 5.2 – Three photos (2.5 MB in total) added and ready to upload to the originals bucket.

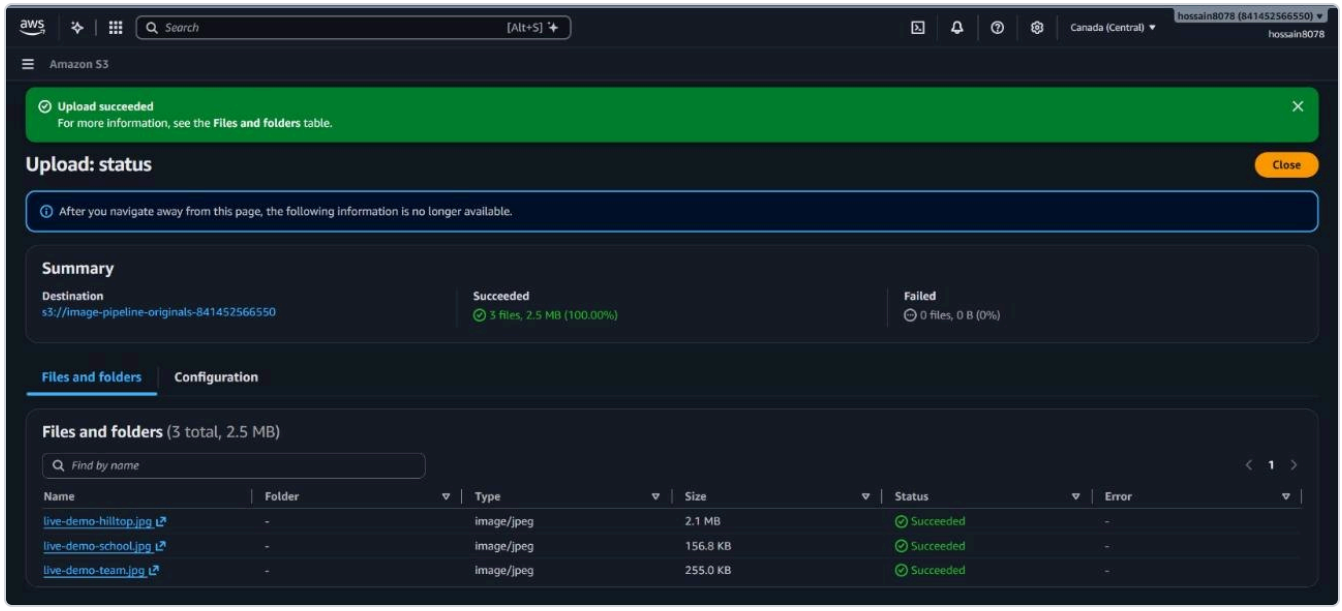


Figure 5.3 – Upload succeeded: 3 files. We click nothing else; S3 now sends an Object Created event for each file.

5.2 Watch the workflow start by itself

1. We open **Step Functions → State machines → image-pipeline-image-processing**.
2. The Executions list shows one new run per uploaded image, already Succeeded.
3. We click the newest run, then the **ResizeImage** step, to see its input and output.

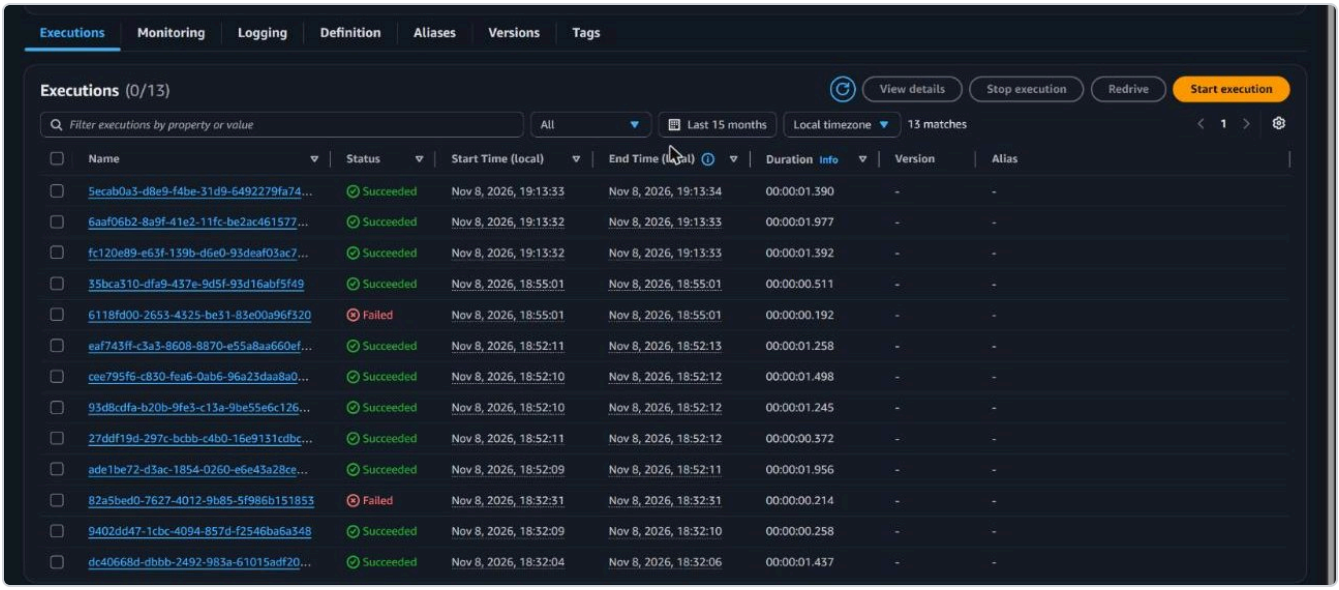


Figure 5.4 – Three new executions started at 19:13:32–33, one per photo, all Succeeded in under 2 seconds.

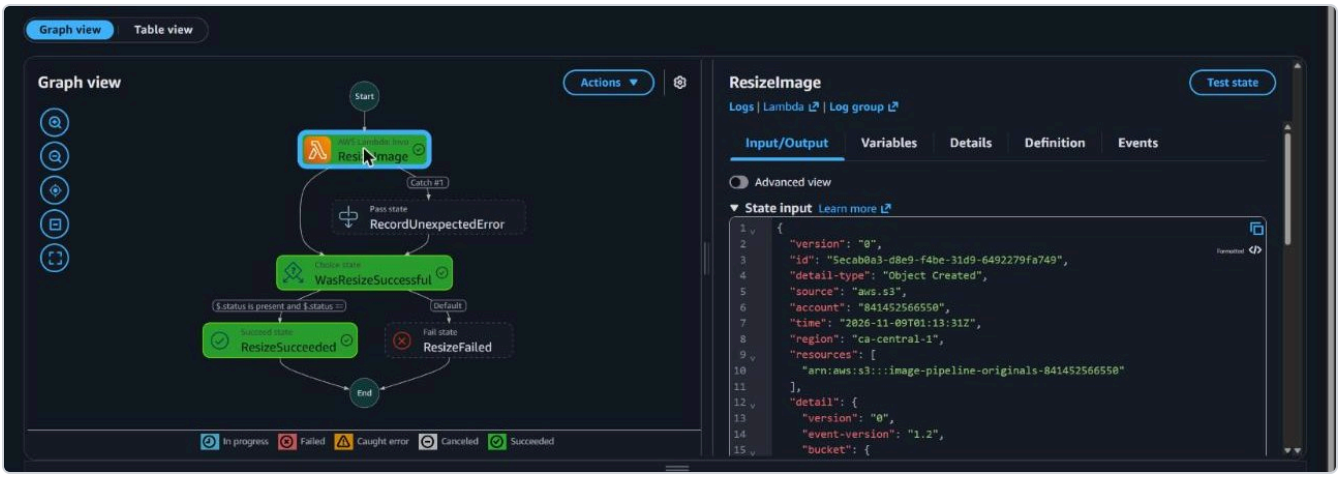


Figure 5.5 – Inside one run: the green path Task → Choice → Succeed; the input is the S3 "Object Created" event sent through EventBridge.

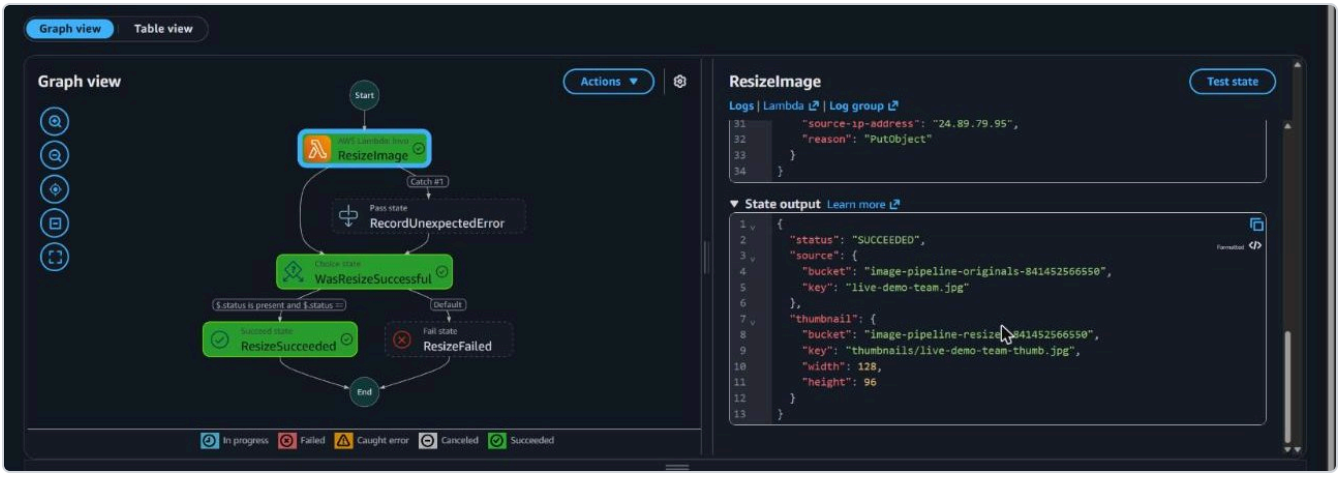


Figure 5.6 – The Lambda output: status SUCCEEDED and the thumbnail thumbnails/live-demo-team-thumb.jpg, 128×96 pixels.

5.3 Check the thumbnails

1. We open **S3 → image-pipeline-resized-<account-id> → thumbnails/**.
2. Each upload has a matching `<name>-thumb.jpg`, a few KB in size.
3. We tick one and click **Open** to view it in a new tab.
4. CLI alternative: `aws s3 ls s3://<ResizedBucketName>/thumbnails/`

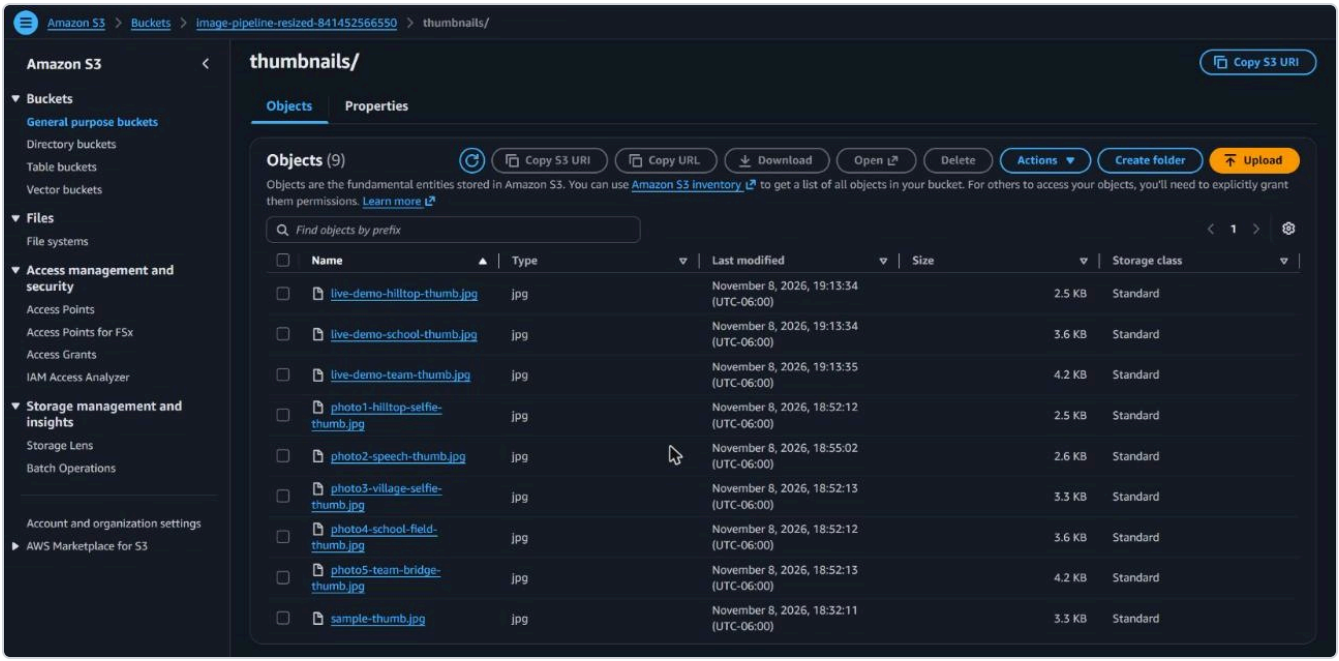


Figure 5.7 – The resized bucket now holds the three new thumbnails, 2.5–4.2 KB each.



Figure 5.8 – A thumbnail opened from S3: the browser tab reads 128×96, down from a 1233×925, 255 KB original.

Chapter 6 – Test the API trigger and the failure path

A POST to `/resize` starts the same workflow: a real image ends in Succeed, and a missing file ends in Fail through the Choice state. This is README test Option B plus "See the failure path". The image must already be in the originals bucket.

6.1 Call the API with a real image

In CloudShell we read the API URL from the stack, then send the request:

```
API=$(aws cloudformation describe-stacks --stack-name image-pipeline --query "Stacks[0].Ou
tputs[?OutputKey=='ResizeApiUrl'].OutputValue" --output text)
curl -s -X POST "$API" -H 'Content-Type: application/json' -d '{"key": "live-demo-school.j
pg"}'
```

PowerShell version:

```
Invoke-RestMethod -Method Post -Uri "<ResizeApiUrl>" -ContentType "application/json" -Body
'{"key": "live-demo-school.jpg"}'
```

The API answers right away with an `executionArn`: the run itself happens in the background.

6.2 Call the API with a missing file

```
curl -s -X POST "$API" -H 'Content-Type: application/json' -d '{"key": "no-such-photo.jp
g"}'
aws stepfunctions list-executions --state-machine-arn <StateMachineArn> --max-items 5 --qu
ery 'executions[].[startDate,status]' --output table
```



Figure 6.1 – CloudShell: the API URL, two POST /resize calls with their execution ARNs, and the run list: the real image SUCCEEDED, the missing file FAILED.

6.3 Inspect the failed run

1. In Step Functions we open the newest run marked **Failed**.
2. The banner reads **ImageResizeFailed in step: ResizeFailed**.

3. We click the **ResizeImage** step to see why.

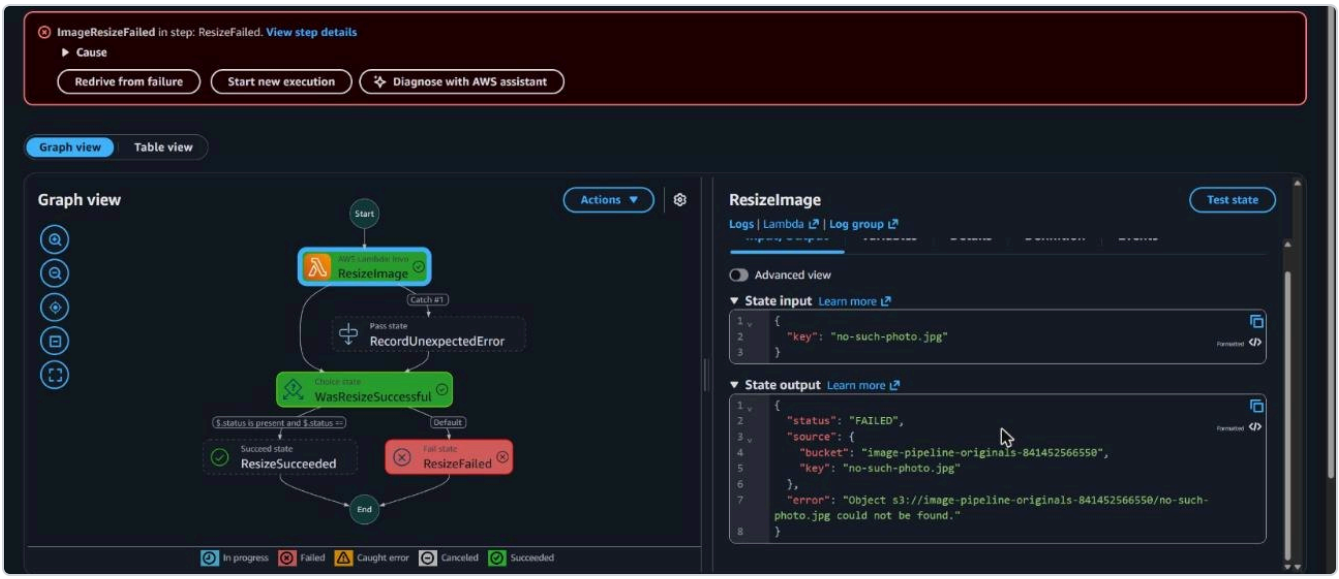


Figure 6.2 – The failed run: Task → Choice → ResizeFailed (red). The Lambda output says status FAILED, "no-such-photo.jpg could not be found".

The Lambda reports expected problems (missing file, corrupt image, unsupported type, wrong bucket) as status FAILED instead of crashing, so the Choice state can route them to Fail. Unexpected errors such as timeouts are caught by the Task's Catch and take the same path.

Chapter 7 – Monitor and check the configuration

Every resize leaves a log line in CloudWatch, and the console shows exactly how API Gateway and the Lambda are wired. This matches the README's Monitoring section.

7.1 CloudWatch logs

1. We open **CloudWatch → Log management → /aws/lambda/image-pipeline-resize-image**.
2. We filter with `"Created s3"` to see only finished thumbnails, and expand a row to read it in full.
3. From CloudShell or a terminal: `sam logs --stack-name image-pipeline --name ResizeImageFunction --tail`

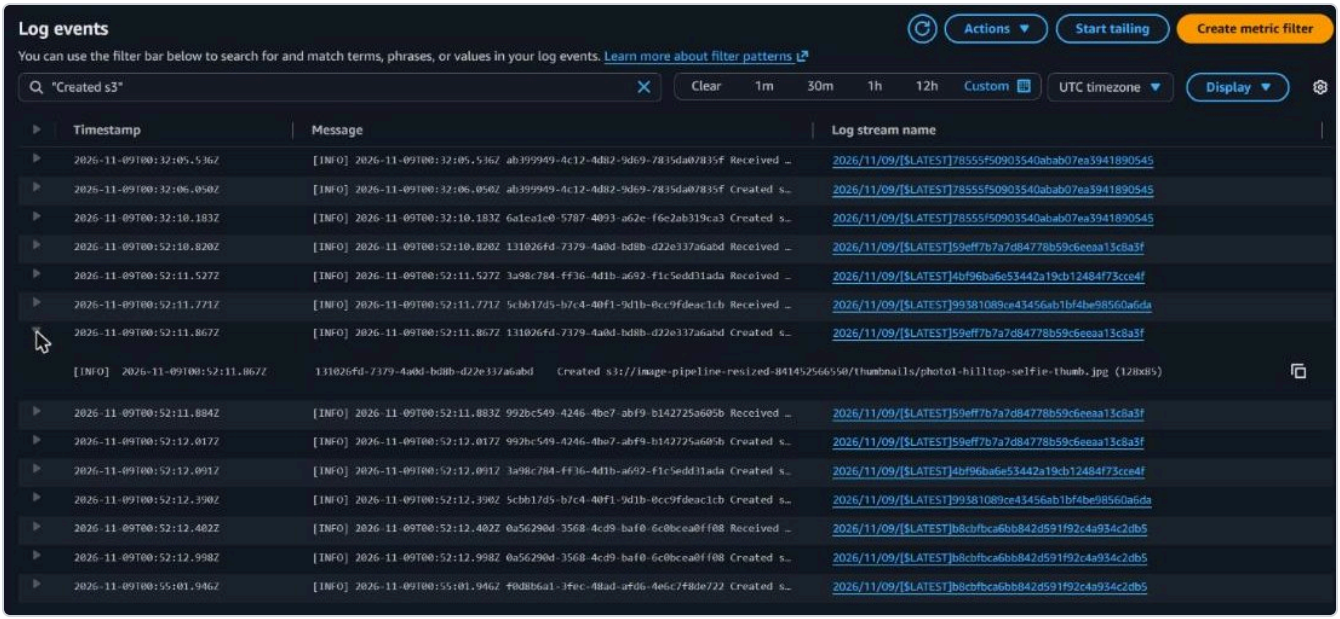


Figure 7.1 – Lambda log events filtered on "Created s3"; the expanded row shows photo1-hilltop-selfie-thumb.jpg created at 128×85.

Where	What we find
/aws/lambda/image-pipeline-resize-image	Each resize: input, output key, size and any error
/aws/vendedlogs/states/image-pipeline-image-processing	Every state transition with its data
Step Functions console	Visual history and the path each run took
AWS X-Ray	Traces across Step Functions and Lambda

7.2 API Gateway configuration

We open **API Gateway → APIs → image-pipeline → Resources → /resize → POST → Integration request**. The integration type is AWS, the service is Step Functions and the action is StartExecution,

so no extra code is needed to start the workflow.

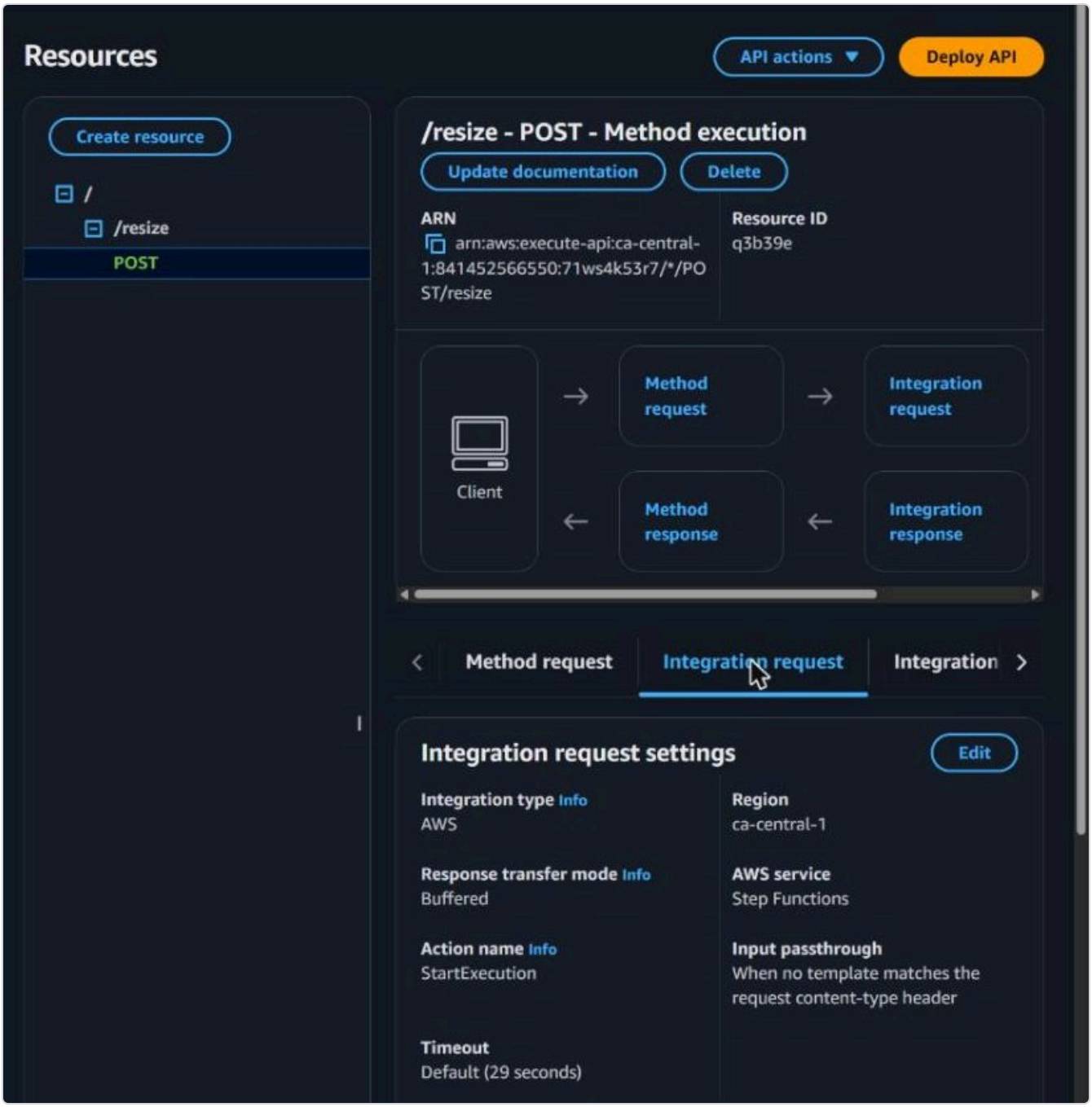


Figure 7.2 – POST /resize integration request: AWS service integration, Step Functions, action StartExecution, region ca-central-1.

7.3 Lambda function

We open **Lambda → Functions → image-pipeline-resize-image**. The function shows Layers (1) for Pillow, and the code tab shows app.py from the repository.

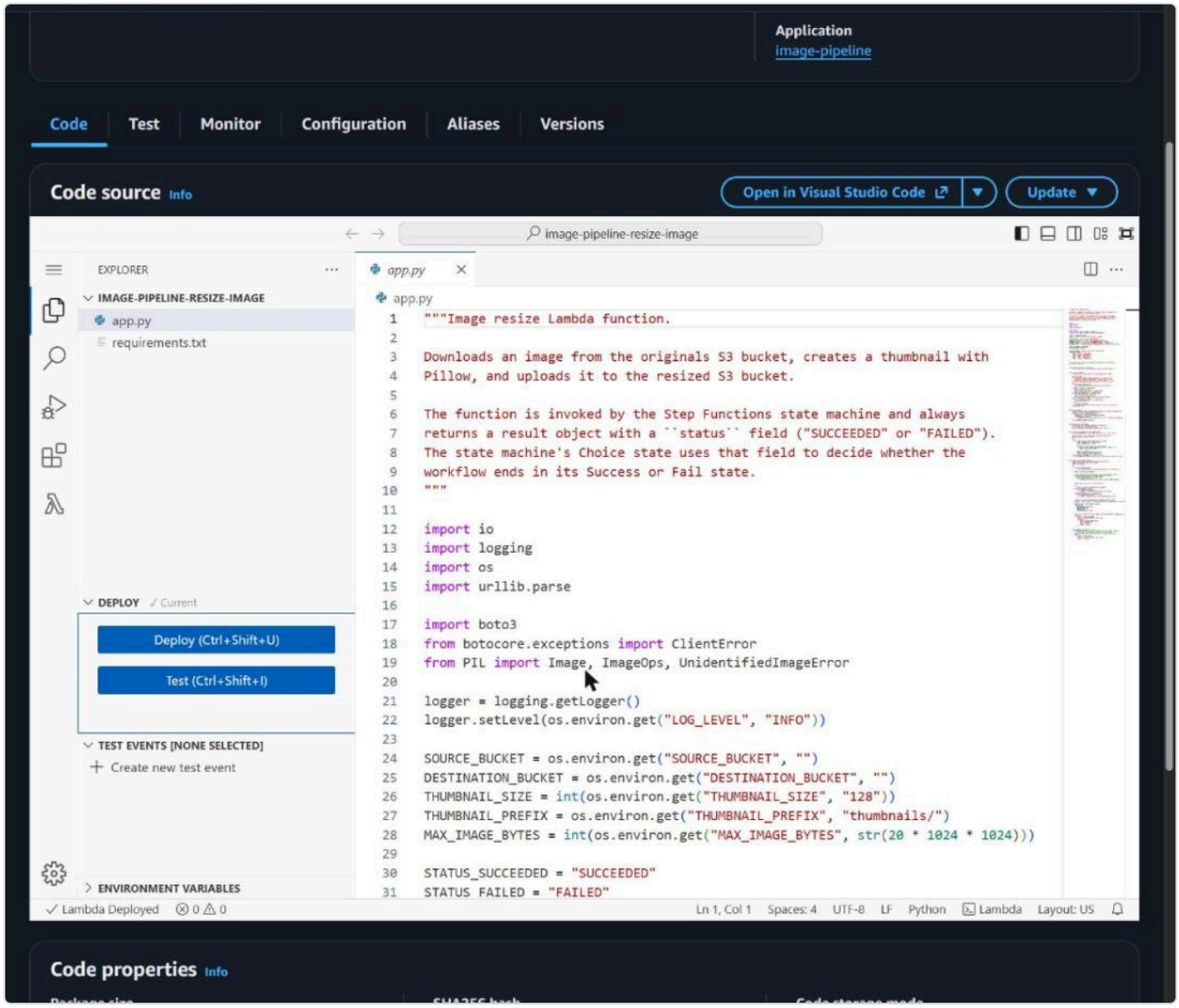


Figure 7.3 – The resize Lambda in the console: app.py imports boto3 and Pillow (from the layer); the function belongs to the image-pipeline application.

The Lambda has least-privilege access: it can only read the originals bucket and write to the resized bucket. Both buckets block all public access and use server-side encryption.

Chapter 8 – Results

The pipeline worked on every real image and failed cleanly on every missing one: each original became a 128-pixel thumbnail of 2.5–4.2 KB, as small as 0.1% of the original size.

We can watch the whole run in the demo video on YouTube. The same evidence is stored in the repository under docs/results.

8.1 Test runs

Test	Trigger	Result
sample.jpg (1600×1200, 123 KB)	S3 upload	Succeeded: 128×96, 3.3 KB
Five personal photos	S3 upload	5 of 5 Succeeded
Three live-demo photos	S3 upload	3 of 3 Succeeded in under 2 seconds
POST /resize with a real image	API Gateway	Succeeded
POST /resize with a missing file	API Gateway	Failed as designed, through the Choice state

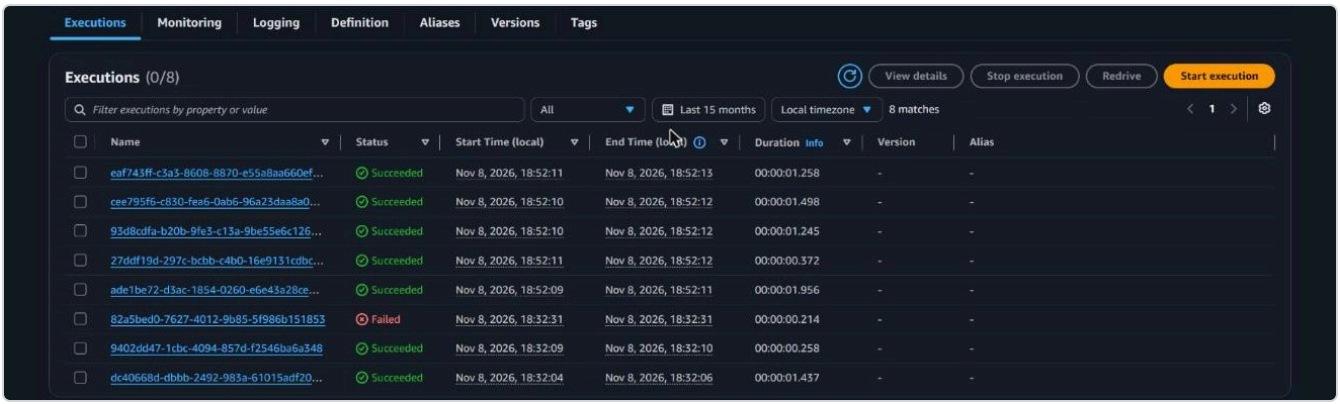


Figure 8.1 – The execution list after the five-photo test: five automatic runs Succeeded next to the earlier test runs, including one designed failure.

8.2 Originals and thumbnails

I downloaded both buckets in CloudShell and measured every file:

Image	Original (px)	Original size	Thumbnail (px)	Thumbnail size
photo1-hilltop-selfie.jpg	3456×2304	2.1 MB	128×85	2.5 KB
photo2-speech.jpg	2048×1365	161 KB	128×85	2.6 KB
photo3-village-selfie.jpg	1440×812	242 KB	128×72	3.3 KB
photo4-school-field.jpg	1440×1080	156 KB	128×96	3.6 KB

Image	Original (px)	Original size	Thumbnail (px)	Thumbnail size
photo5-team-bridge.jpg	1233×925	255 KB	128×96	4.2 KB

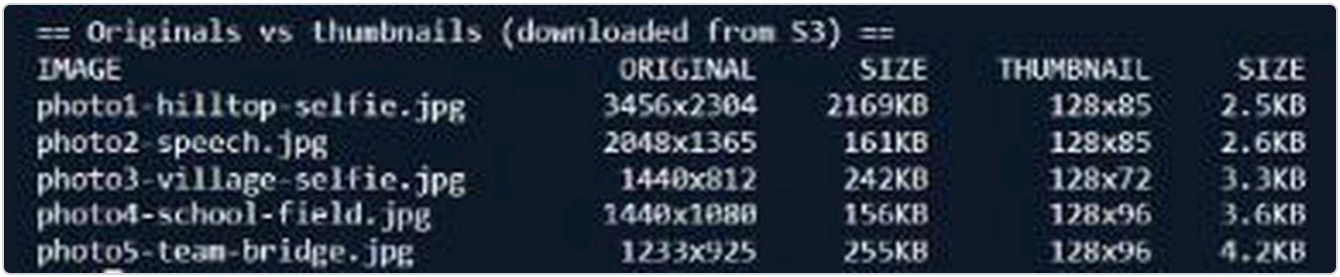


Figure 8.2 – CloudShell measurement of the downloaded files: every original reduced to 128 px on its longest side.

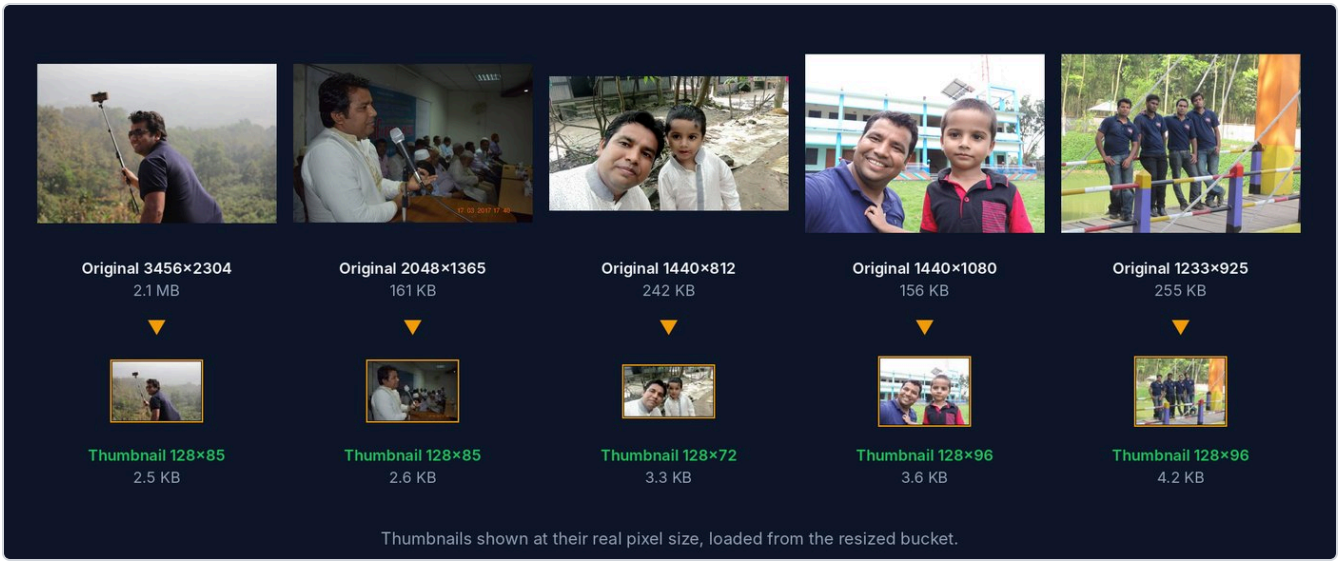


Figure 8.3 – Originals beside their real thumbnails from the resized bucket: aspect ratio kept, longest side 128 px.

8.3 Requirements checklist

Requirement	Status
S3 bucket for original images	✓ Met
Lambda runs when a new image is uploaded	✓ Met
Lambda resizes the image to a thumbnail with Pillow	✓ Met
Thumbnail stored in a second S3 bucket	✓ Met
Step Functions workflow with Task, Choice, Succeed and Fail states	✓ Met
API Gateway endpoint triggers the workflow	✓ Met
Source code and README on GitHub, plus 18 passing unit tests	✓ Met

Chapter 9 – Troubleshooting and clean up

Most problems come from the five causes listed below, and one three-command clean-up removes everything when we are done.

9.1 Common problems

Problem	Fix
"Bucket name ... is not valid" on deploy	Use a lowercase stack name, e.g. image-pipeline
"No module named 'PIL'" in the Lambda logs	Rebuild with `sam build --use-container`, then `sam deploy`
No thumbnail after an upload	Check the file type (JPG, JPEG, PNG, GIF, WEBP), then open the latest Step Functions run
API returns an executionArn but no thumbnail	The API starts the run in the background; open that run in Step Functions
CloudShell says the session ended	Click **Reconnect**; files in the home folder are kept

9.2 Configuration we can change

Setting	Where	Default
Thumbnail size (longest side, px)	ThumbnailSize parameter	128
Thumbnail key prefix	THUMBNAIL_PREFIX environment variable	thumbnails/
Maximum image size	MAX_IMAGE_BYTES environment variable	20 MB

Example: `sam deploy --parameter-overrides ThumbnailSize=256`

9.3 Clean up

S3 buckets must be empty before CloudFormation can delete them, so we empty them first:

```
aws s3 rm s3://<OriginalsBucketName> --recursive
aws s3 rm s3://<ResizedBucketName> --recursive
sam delete --stack-name image-pipeline
```

This removes the buckets, Lambda, state machine, API, EventBridge rule, roles and log groups, so nothing keeps running or costing money.